

Relatório EmbarcaTech

Projeto SafeDriver IoT

Aluno: Guilherme Barros Vieira de Araujo

Introdução ao projeto

O SafeDriver é um carrinho robótico inteligente desenvolvido para operar de forma autônoma ou ser controlado remotamente via WiFi. Baseado na Raspberry Pi Pico W, o sistema utiliza um sensor ultrassônico para detectar obstáculos e tomar decisões em tempo real, desviando-se automaticamente de colisões. Além disso, o carrinho pode ser controlado manualmente por meio de uma interface web acessível pelo navegador, sem a necessidade de aplicativos externos. O projeto utiliza um chassi 4WD com quatro motores DC.



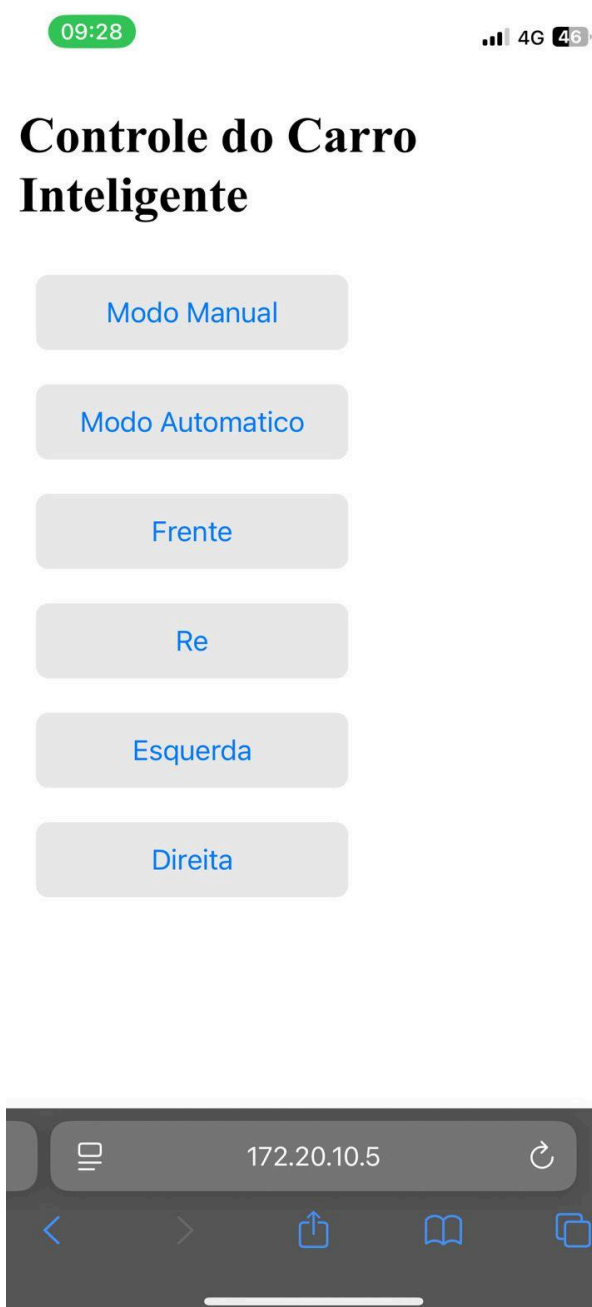
O desenvolvimento do SafeDriver se justifica por sua aplicação em aprendizado de sistemas embarcados, integrando controle de motores, sensores e comunicação via WiFi. Além disso, ele permite explorar conceitos de navegação autônoma, proporcionando um ambiente acessível para testar algoritmos de movimentação e automação. Sua modularidade permite futuras expansões, como a adição de novos sensores e estratégias mais avançadas de controle.

Embora existam projetos similares de carrinhos robóticos, o SafeDriver se diferencia pelo uso da Raspberry Pi Pico W, que combina conectividade WiFi eficiente com baixo consumo de energia, e pelo controle de curvas aprimorado, otimizando a movimentação em espaços reduzidos. A interface web torna seu uso mais intuitivo e acessível, permitindo que qualquer dispositivo conectado à rede possa interagir com o robô de forma prática.

O projeto combina simplicidade e eficiência, utilizando componentes acessíveis e um código modular programado na Arduino IDE. Com isso, o SafeDriver se torna uma excelente plataforma para estudantes, entusiastas e pesquisadores interessados em robótica, automação e sistemas embarcados, unindo controle remoto, navegação inteligente e otimização de hardware em um único projeto funcional.

Descrição do funcionamento

O SafeDriver possui dois modos de operação principais: modo manual e modo automático, ambos acessíveis por meio de uma interface web simples e intuitiva. Essa interface permite ao usuário interagir com o carrinho de forma remota, bastando que o dispositivo esteja conectado à mesma rede WiFi da Raspberry Pi Pico W.



No modo manual, o controle é feito por comandos enviados diretamente pelo usuário a partir da interface web. A página de controle apresenta botões para as ações principais, como Frente, Ré, Esquerda, Direita e Parar. Cada comando é enviado instantaneamente para a Raspberry Pi Pico W, que executa a movimentação correspondente ao comando recebido. Esse modo oferece total controle sobre o carrinho, sendo ideal para testes de movimentação e exploração do ambiente.

No modo automático, o carrinho opera de forma independente, utilizando um sensor ultrassônico para detectar obstáculos no caminho. Quando um obstáculo é identificado, o carrinho executa uma manobra de desvio, alterando sua direção para continuar o percurso sem colisões. Esse modo é gerido por um algoritmo que processa as informações do sensor e toma decisões em tempo real, garantindo a navegação segura e eficiente.

O sistema de controle de motores, baseado na ponte H L298N, permite ao carrinho executar movimentos precisos em qualquer direção. Além disso, foi implementada uma lógica de controle

otimizada para curvas, onde os motores do lado oposto à direção da curva giram no sentido contrário, aumentando a força e eficiência das manobras. Com isso, o SafeDriver oferece tanto uma experiência interativa no modo manual quanto uma operação autônoma confiável no modo automático.

Especificação do Hardware

A especificação de hardware do projeto SafeDriver descreve os componentes utilizados para o funcionamento do carrinho robótico, detalhando suas conexões e funções. O sistema é composto pela Raspberry Pi Pico W, responsável pelo controle geral e pela comunicação WiFi; uma ponte H L298N, que gerencia a direção e o sentido de rotação dos quatro motores DC; e um sensor ultrassônico, usado para detectar obstáculos no caminho do carrinho. Esses componentes são alimentados por uma fonte de energia externa, garantindo tensão e corrente suficientes para o funcionamento eficiente do sistema. O hardware foi projetado para ser modular, acessível e otimizado para aplicações de robótica móvel.

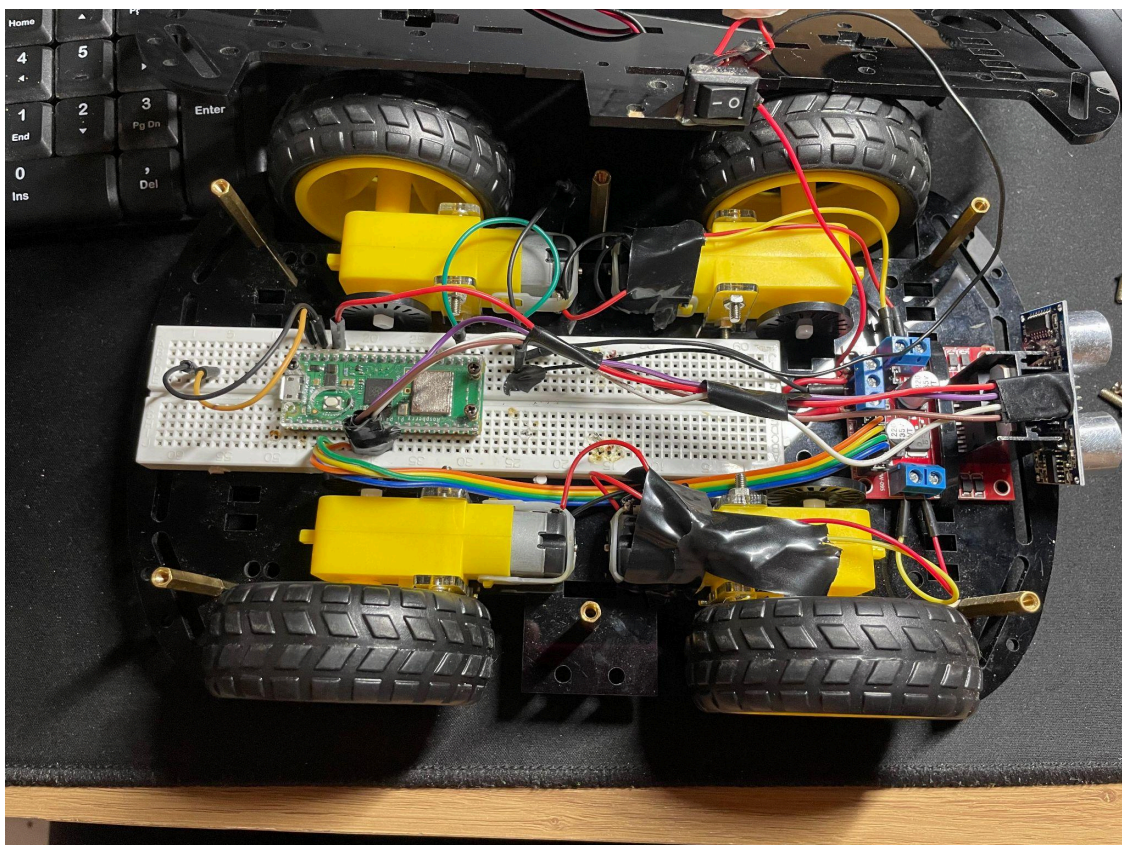
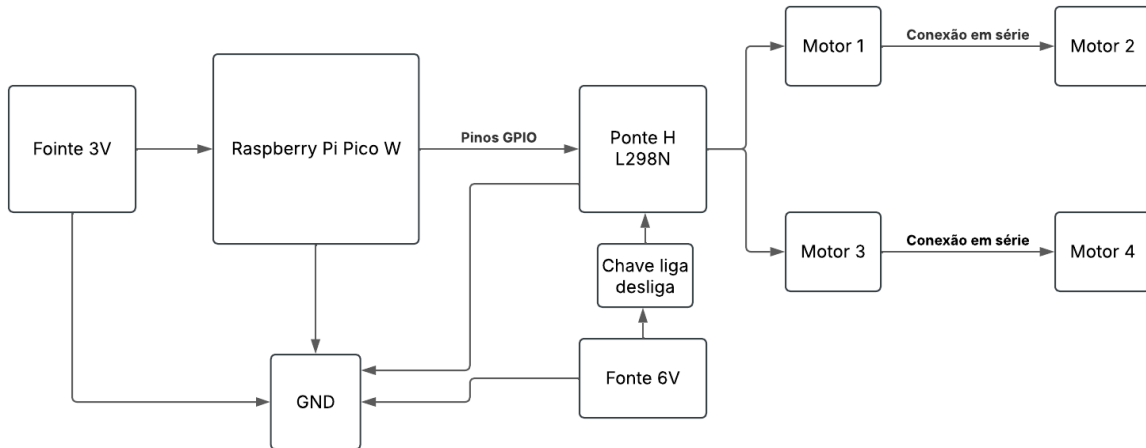


Diagrama de blocos do hardware

Especificação de Hardware



Função e Configuração de Cada Bloco

- **Raspberry Pi Pico W**

- **Função:** Controla o sistema, gerencia os sinais para a ponte H, processa os dados do sensor ultrassônico e realiza a comunicação WiFi.
- **Configuração:** Configurada como WiFi Station; GPIOs configurados como saída para controle da ponte H.
- **Comandos e Registros:**

Inicialização do WiFi:

```
WiFi.begin(ssid, password);
```

Configuração do servidor HTTP:

```
WiFiServer server(80);
```

Configuração inicial dos GPIOs:

```
pinMode(MOTOR_ESQUERDO_FRENTE_PIN, OUTPUT);
```

```
pinMode(MOTOR_ESQUERDO_RE_PIN, OUTPUT);
```

```
pinMode(MOTOR_DIREITO_FRENTE_PIN, OUTPUT);
```

```
pinMode(MOTOR_DIREITO_RE_PIN, OUTPUT);
```

- **Ponte H L298N**

- **Função:** Controla o sentido e a direção dos motores DC.

- **Configuração:** Conectada aos GPIOs da Pico W para controle; recebe alimentação de 6V.
- **Comandos e Registros:**
 - Controle dos motores:

Para mover para frente:

```
digitalWrite(MOTOR_ESQUERDO_FRENTE_PIN, HIGH);
digitalWrite(MOTOR_DIREITO_FRENTE_PIN, HIGH);
digitalWrite(MOTOR_ESQUERDO_RE_PIN, LOW);
digitalWrite(MOTOR_DIREITO_RE_PIN, LOW);
```

- **Motores DC**
 - **Função:** Realizam a movimentação do carrinho.
 - **Configuração:** Dois pares de motores ligados em série, conectados à ponte H.
- **Fonte de 3V**
 - **Função:** Alimenta a Raspberry Pi Pico W.
 - **Configuração:** Conectada diretamente à placa.
- **Fonte de 6V**
 - **Função:** Alimenta a ponte H e os motores DC.
 - **Configuração:** Conectada à ponte H por meio de uma chave liga/desliga.
- **Ground (GND)**
 - **Função:** Fornece uma referência elétrica comum para todos os componentes.
 - **Configuração:** Conectado ao pino 23 da Raspberry Pi Pico W, compartilhado por todos os dispositivos.
- **Sensor Ultrassônico**
 - **Função:** Detecta obstáculos à frente e envia dados de distância para a Raspberry Pi Pico W.
 - **Configuração:** Conectado aos GPIOs da Pico W, utilizando um pino para o trigger e outro para o echo.
 - **Comandos e Registros:**

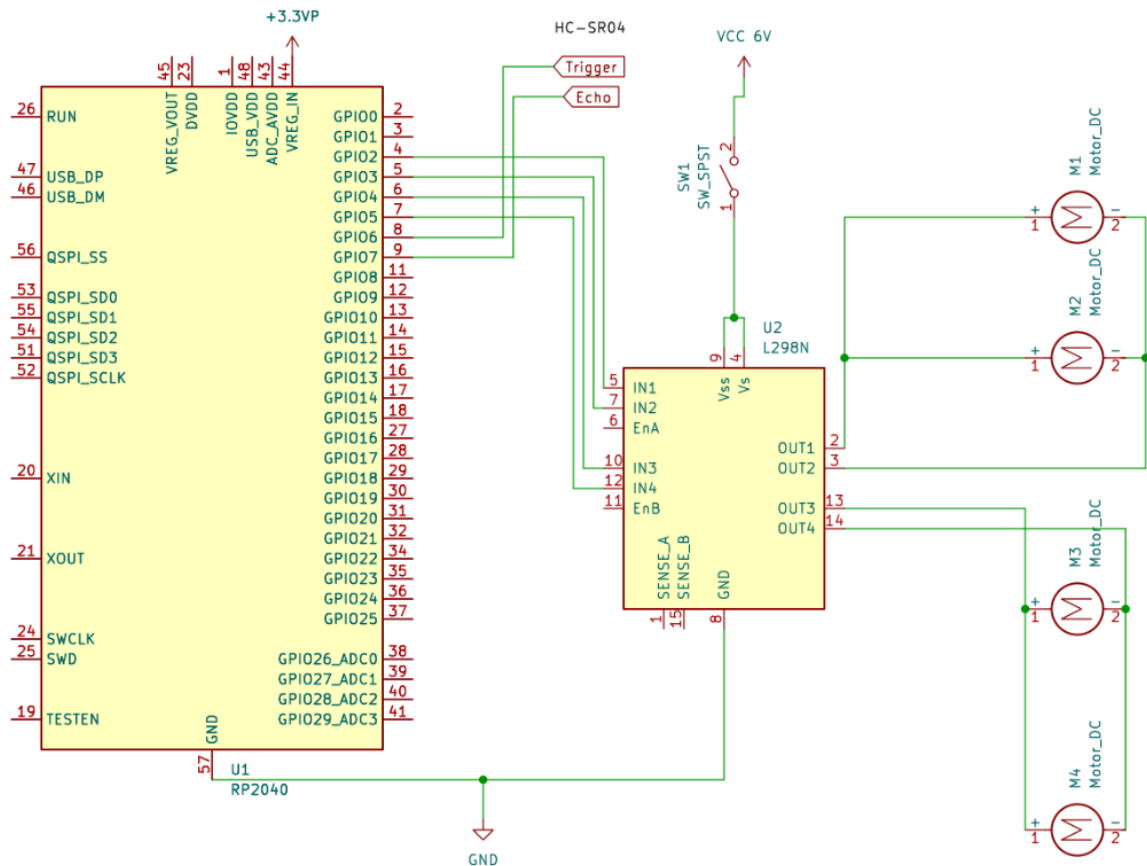
Pulso no pino de trigger:

```
digitalWrite(TRIGGER_PIN, HIGH);
delayMicroseconds(10);
digitalWrite(TRIGGER_PIN, LOW);
```

Leitura do sinal de echo:

```
long duration = pulseIn(ECHO_PIN, HIGH);
float distance = (duration * 0.034) / 2;
```

Circuito completo do Hardware

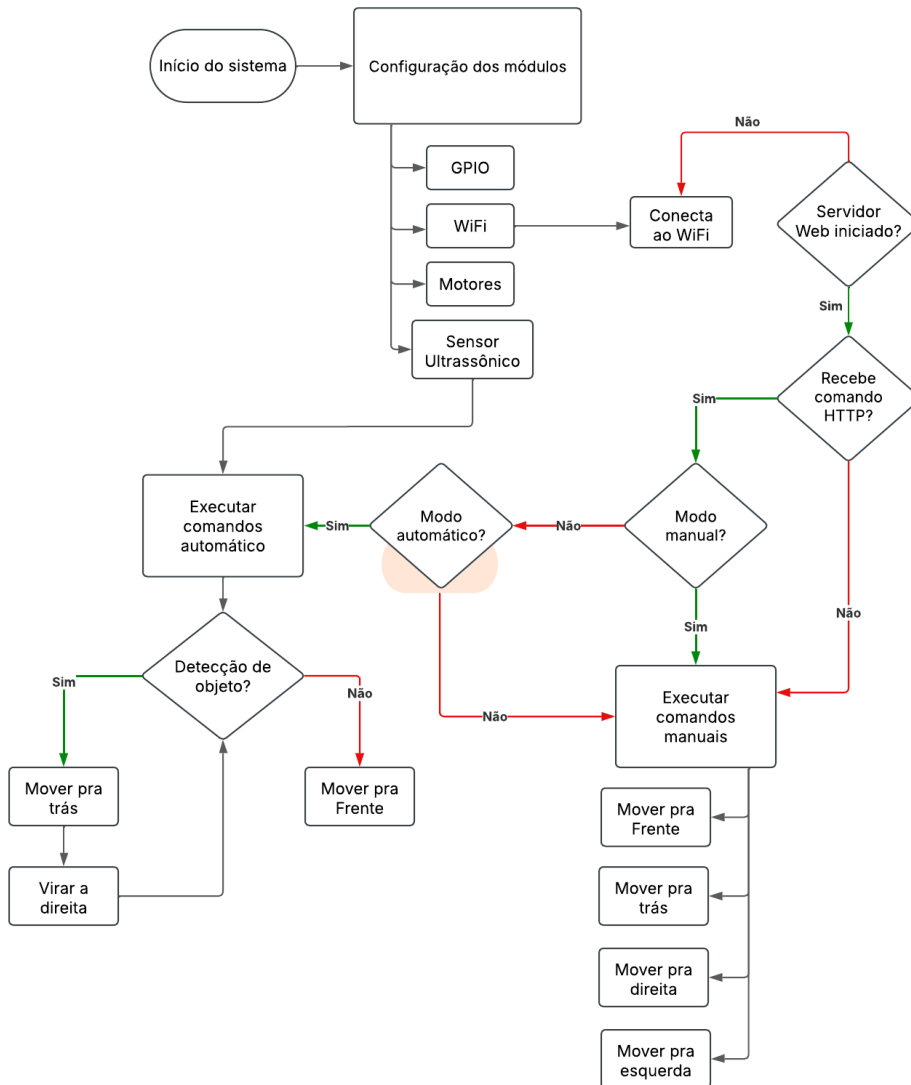


Especificação do Firmware

O firmware do projeto SafeDriver foi desenvolvido para integrar o hardware e a lógica de controle do carrinho robótico de forma eficiente. Ele combina comunicação WiFi para controle manual via interface web com o uso de um sensor ultrassônico para habilitar o modo automático, onde o carrinho identifica e evita obstáculos. A organização modular do software facilita a manutenção e a escalabilidade, permitindo configurar pinos GPIO, gerenciar motores, e realizar leituras precisas do sensor. Além disso, o processo de inicialização garante que todas as funções essenciais, como a conexão WiFi e o servidor HTTP, estejam prontas para operação, assegurando uma transição fluida entre os modos de controle manual e automático.

Diagrama de blocos do hardware

Especificação de Hardware



Descrição das Funcionalidades

- **Bloco de Configuração de Módulos**
 - Configura os GPIOs para controle dos motores e leitura do sensor ultrassônico.
 - Inicializa a comunicação WiFi e o servidor HTTP para controle remoto.
- **Bloco de Comunicação (HTTP)**
 - Recebe comandos via navegador (HTTP GET) e os processa para acionar os motores no modo manual.
- **Bloco de Controle de Movimento**

- Controla os motores com base nos comandos recebidos (modo manual) ou na leitura do sensor ultrassônico (modo automático).
- **Bloco de Lógica do Modo Automático**
 - Mede a distância usando o sensor ultrassônico e decide se o carrinho deve continuar para frente ou realizar uma manobra de meia-volta para evitar obstáculos.
- **Definição das variáveis**

Variável	Tipo	Descrição
isManualMode	bool	indica se o carrinho está em modo manual ou automático.
isMovingForward	bool	Indica se o carrinho está se movendo para frente.
header	String	Armazena o cabeçalho da requisição HTTP recebida pelo servidor.
ssid e password	const char*	Credenciais de conexão WiFi
TRIGGER_PIN, ECHO_PIN	#define	Pinos do sensor ultrassônico para medir a distância
MOTOR_*_PIN	#define	Pinos GPIO para controlar os motores

- **Inicialização**

O processo de inicialização é realizado no método `setup()`

1. **Configura os pinos GPIO como entrada ou saída**

Configura os pinos GPIO como entrada ou saída:

```
pinMode(TRIGGER_PIN, OUTPUT);
```

```
pinMode(ECHO_PIN, INPUT);
```

```
pinMode(MOTOR_ESQUERDO_FRENTE_PIN, OUTPUT);
```

```
pinMode(MOTOR_DIREITO_FRENTE_PIN, OUTPUT);
```

```
pinMode(MOTOR_ESQUERDO_RE_PIN, OUTPUT);
```

```
pinMode(MOTOR_DIREITO_RE_PIN, OUTPUT);
```

2. Inicializa o monitor serial para depuração:
`Serial.begin(115200);`
3. Estabelece a conexão WiFi:
`WiFi.begin(ssid, password);`
4. Inicia o servidor HTTP para controle remoto:
`server.begin();`

- **Configurações dos Registros**

Os registros GPIO são configurados com `pinMode` para indicar a direção do pino (entrada ou saída). Os motores e o sensor ultrassônico são controlados diretamente com `digitalWrite` e `pulseIn`.

1. Estrutura e Formato dos Dados

- Sensor Ultrassônico:

Mede a distância em centímetros:

```
float distance = (duration * 0.034) / 2;
```

- Comandos HTTP:

- São strings recebidas no formato `GET /<comando>` (ex.: `GET /forward`).

- **Organização da Memória**

- Globais: Variáveis como `isManualMode` e header armazenam o estado atual do carrinho e os dados da requisição.
- Locais: Dados temporários, como o valor da distância lida pelo sensor ultrassônico, são mantidos em variáveis locais dentro de funções.
-

- **Protocolo de Comunicação**

O firmware utiliza o protocolo HTTP para comunicação entre o navegador do usuário e a Raspberry Pi Pico W. Comandos são enviados no formato de requisições HTTP GET e processados pelo servidor.

- **Formato do Pacote de Dados**

Os pacotes de dados são representados por requisições HTTP GET, com o seguinte formato:

- `GET /manual` – Ativa o modo manual.
- `GET /auto` – Ativa o modo automático.
- `GET /forward` – Move o carrinho para frente.
- `GET /stop` – Para o carrinho.
- Outros comandos (`/backward`, `/left`, `/right`) seguem o mesmo padrão.

Execução do projeto

Metodologia

O desenvolvimento do projeto seguiu uma abordagem estruturada, dividida em algumas etapas:

1. Pesquisas Realizadas:

- Estudamos soluções existentes em robótica e projetos similares que utilizam sensores ultrassônicos e controle por WiFi, garantindo uma base teórica sólida.
- Analisamos modelos de hardware disponíveis, considerando custo, funcionalidade e integração com software.

2. Escolha do Hardware:

- Optamos pela Raspberry Pi Pico W devido ao seu suporte ao WiFi, simplicidade de programação e boa relação custo-benefício.
- Utilizamos o chassi 4WD com ponte H L298N, escolhido por sua robustez e compatibilidade com motores DC.
- Complementamos o sistema com uma fonte de 3V para a placa e 6V para os motores, garantindo alimentação independente e estável.

3. Definição das Funcionalidades do Software:

- Implementamos dois modos de operação: manual e automático.
- Desenvolvemos o controle via interface web para o modo manual e o desvio de obstáculos no modo automático, utilizando o sensor ultrassônico.

4. Inicialização da IDE:

- Configuramos o ambiente de desenvolvimento com a Arduino IDE no Linux.
- Adicionamos as bibliotecas necessárias, como suporte ao WiFi e comunicação HTTP.

5. Programação na IDE:

- Codificamos as funcionalidades principais, como o controle dos motores, leitura do sensor ultrassônico e interface web.
- Implementamos a lógica de decisão para o modo automático, priorizando segurança e eficiência.

6. Depuração:

- Utilizamos o monitor serial para depurar erros na comunicação WiFi e no controle dos motores.

- Realizamos ajustes finos na temporização do sensor ultrassônico e na força aplicada às rodas para otimizar curvas e movimentos.

Testes de Validação

1. Teste do Sensor Ultrassônico:

- Validamos a precisão da leitura de distâncias em ambientes com diferentes níveis de iluminação e obstáculos de tamanhos variados.

2. Teste dos Motores:

- Avaliamos a capacidade dos motores de realizar movimentos para frente, ré e curvas em diferentes superfícies, ajustando a lógica para curvas mais eficientes.

3. Teste do Modo Manual:

- Controlamos o carrinho remotamente por meio da interface web, garantindo resposta imediata aos comandos de movimentação.

4. Teste do Modo Automático:

- Simulamos cenários com obstáculos a diferentes distâncias para verificar a capacidade do carrinho de evitar colisões e executar manobras de meia-volta.

5. Teste de Conectividade WiFi:

- Avaliamos a estabilidade da comunicação em distâncias variadas, garantindo a funcionalidade do servidor HTTP em diferentes condições de sinal.

Discussão dos Resultados

Os testes realizados demonstraram que o projeto SafeDriver é funcional e confiável dentro das especificações propostas. A interface web apresentou resposta rápida e eficiente aos comandos, permitindo controle total do carrinho em modo manual. No modo automático, o sensor ultrassônico foi capaz de detectar obstáculos com precisão, e a lógica de movimentação executou as manobras esperadas.



Apesar do sucesso geral, observamos que a alimentação dos motores limita o desempenho em superfícies mais rugosas. Isso indica que uma futura melhoria seria o uso de uma fonte de alimentação mais robusta para os motores ou a substituição dos motores DC por modelos mais potentes.

O projeto SafeDriver atendeu aos objetivos propostos, sendo capaz de operar de forma autônoma e controlada, mostrando-se uma solução viável para experimentação e aprendizado em robótica móvel. Em uma futura versão pretendemos incluir a utilização de um sistema de tempo real como FreeRTOS para que seja possível criar tasks e garantir melhores desempenhos por meio de um SO. Também queremos integrar ao Hardware sensores mais potentes e precisos para garantir maior autonomia do carro quando estiver no modo automático.